Ad Soyad: _____	**Tarih:** _____
Lab 01 – Hafta 1 – *STL Kapsayıcılara Giriş*	

Lab Hakkında

Bu laboratuvar çalışmasında STL kapsayıcılarını uygulamalı olarak kullanacaksınız. Her çalışma bölümünde önce örnek kodu inceleyip çalıştırın, ardından görevleri tamamlayın.
Süre: 2 saat **Derleme:** `g++ -std=c++20 -o program dosya.cpp`

1 Çalışma 1: vector ile Öğrenci Listesi (25 dk)

Ön Bilgi

`std::vector` dinamik boyutlu, ardışık bellekte saklanan dizidir. Eleman ekleme (`push_back`) amortize $O(1)$, indeks erişimi $O(1)$'dir.

Aşağıdaki başlangıç kodunu `calisma1.cpp` dosyasına yazın ve derleyin:

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

int main() {
    std::vector<std::string> ogrenciler;

    // Ogrenci ekle
    ogrenciler.push_back("Ali Yilmaz");
    ogrenciler.push_back("Ayse Kaya");
    ogrenciler.push_back("Mehmet Demir");
    ogrenciler.emplace_back("Zeynep Can");

    std::cout << "Ogrenci sayisi: " << ogrenciler.size() << '\n';
    std::cout << "Kapasite: " << ogrenciler.capacity() << '\n';

    // Range-based for ile listele
    for (const auto& isim : ogrenciler) {
        std::cout << "  - " << isim << '\n';
    }

    return 0;
}
```

`push_back` vs `emplace_back`

`push_back` nesneyi kopyalar/taşır, `emplace_back` ise doğrudan yerinde inşa eder (in-place construction). String gibi türlerde `emplace_back` daha verimli olabilir.

Görev 1.1 – Kapasite Gözlemi (5 dk)

Programı çalıştırın. Ardından döngü ile 20 öğrenci daha ekleyin (`"Ogrenci_1"`, `"Ogrenci_2"`, ...). Her ekleme sonrası `size()` ve `capacity()` değerlerini yazdırın.
Gözlem: Kapasite hangi anlarda artıyor? Artış oranı nedir?

Görev 1.2 – reserve() ile Optimizasyon (5 dk)

Programın başına `ogrenciler.reserve(30)` ekleyin ve Görev 1.1'i tekrarlayın.
Soru: Yeniden tahsis (reallocation) sayısı nasıl değişti?

Görev 1.3 – Arama ve Silme (10 dk)

Aşağıdaki işlemleri kodunuza ekleyin:

a) `std::find` ile `"Mehmet Demir"` adlı öğrenciyi bulun

b) Bulduğunuz öğrenciyi `erase()` ile silin

c) `at()` ile güvenli erişim yapın ve `out_of_range` istisnasını yakalayın

Dikkat

`erase()` sonrası silinen elemandan sonraki tüm iteratörler geçersiz olur! Döngü içinde silme yapıyorsanız dikkatli olun.

2 Çalışma 2: deque ve Diğer Sıralı Kapsayıcılar (20 dk)

Ön Bilgi

`std::deque` (double-ended queue) hem baştan hem sondan $O(1)$ ekleme/silme destekler. `vector`'den farklı olarak bellekte ardışık değildir – bloklar halinde saklanır.

Aşağıdaki kodu `calisma2.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <deque>
#include <list>
#include <string>
#include <chrono>

// Basit zamanlama fonksiyonu
template <typename Func>
double olc(Func f) {
    auto bas = std::chrono::high_resolution_clock::now();
    f();
    auto son = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> sure = son - bas;
    return sure.count();
}

int main() {
    const int N = 100000;

    // deque: basa ekleme
    std::deque<int> d;
    double t1 = olc([&]() {
        for (int i = 0; i < N; ++i)
            d.push_front(i);
    });
    std::cout << "deque push_front: " << t1 << " ms\n";

    // list: basa ekleme
    std::list<int> l;
    double t2 = olc([&]() {
        for (int i = 0; i < N; ++i)
            l.push_front(i);
    });
    std::cout << "list  push_front: " << t2 << " ms\n";

    return 0;
}
```

Görev 2.1 – vector ile Karşılaştırma (10 dk)

Koda bir de `std::vector` testi ekleyin:

a) `vector`'e **baştan** ekleme: `v.insert(v.begin(), i)`

b) `vector`'e **sondan** ekleme: `v.push_back(i)`

Dört testi karşılaştırın. Neden vector baştan ekleme bu kadar yavaş?

Görev 2.2 – Ortadan Ekleme/Silme (10 dk)

Her üç kapsayıcı için ortadaki elemana erişim ve ortadan silme süresini ölçün.
Tablo olarak kaydedin:

İşlem	vector	deque	list
Baştan ekleme	_____	_____	_____
Sondan ekleme	_____	_____	_____
Ortadan silme	_____	_____	_____

3 Çalışma 3: map ile OBYS Prototipi (30 dk)

Ön Bilgi

`std::map<Key, Value>` anahtar-değer eşlemesi sağlar. Dahili yapısı kırmızı-siyah ağaçtır. Arama, ekleme, silme $O(\log n)$'dir. Anahtarlar her zaman sıralı tutulur.

Aşağıdaki OBYS prototipini `calisma3.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <map>
#include <string>

struct Ogrenci {
    std::string isim;
    double not_ort;
};

int main() {
    std::map<int, Ogrenci> kayit;

    // Ogrenci ekle (numara -> bilgi)
    kayit[2001] = {"Ali Yilmaz", 3.45};
    kayit.insert({2002, {"Ayse Kaya", 3.80}});
    kayit.emplace(2003, Ogrenci{"Mehmet Demir", 2.90});

    // C++17 structured bindings ile dolasma
    for (const auto& [no, ogr] : kayit) {
        std::cout << no << ": " << ogr.isim
                  << " (Ort: " << ogr.not_ort << ")\n";
    }

    // Arama
    if (auto it = kayit.find(2002); it != kayit.end()) {
        std::cout << "\nBulundu: " << it->second.isim << '\n';
    }

    return 0;
}
```

Görev 3.1 – Güvenli Erişim (5 dk)

a) `kayit[9999]` yazıp sonucu gözlemleyin. Ne oldu?

b) `kayit.at(9999)` ile deneyin ve istisnayı yakalayın.

c) `kayit.contains(2001)` (C++20) ile kontrol edin.

Görev 3.2 – Not Güncelleme ve Filtreleme (10 dk)

a) 2001 numaralı öğrencinin not ortalamasını 3.60'a güncelleyin

b) Not ortalaması 3.0'ın üzerindeki öğrencileri listeleyen bir fonksiyon yazın: `void basarililar(const std::map<int, Ogrenci>& kayit)`

Görev 3.3 – set ile Ders Kaydı (15 dk)

`std::set<std::string>` kullanarak bir ders kayıt sistemi oluşturun:

a) Her öğrenciye (`map` içinde) bir `set<string>` ekleyin (kayıtlı dersler)

b) Aynı dersi iki kez eklemeyi deneyin – ne olur?

c) İki öğrencinin ortak derslerini bulan bir fonksiyon yazın (`std::set_intersection` kullanın)

`set_intersection` Kullanımı

```
#include <algorithm>
#include <iterator>

std::set<std::string> ortak;
std::set_intersection(
    a.begin(), a.end(),
    b.begin(), b.end(),
    std::inserter(ortak, ortak.begin())
);
```

4 Çalışma 4: unordered_map ve Uyarlayıcılar (25 dk)

Ön Bilgi

`std::unordered_map` hash tablosu kullanır – ortalama $O(1)$ arama! Ancak elemanlar sırasızdır. `stack`, `queue` ve `priority_queue` ise mevcut kapsayıcıları kısıtlayan uyarlayıcılardır.

Aşağıdaki kodu `calisma4.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <map>
#include <unordered_map>
#include <stack>
#include <string>
#include <chrono>

int main() {
    const int N = 500000;

    // map vs unordered_map arama karsilastirmasi
    std::map<int, int> m;
    std::unordered_map<int, int> um;

    for (int i = 0; i < N; ++i) {
        m[i] = i * 2;
        um[i] = i * 2;
    }

    // map arama
    auto t1 = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < N; ++i) m.find(i);
    auto t2 = std::chrono::high_resolution_clock::now();

    // unordered_map arama
    auto t3 = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < N; ++i) um.find(i);
    auto t4 = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> d1 = t2 - t1;
    std::chrono::duration<double, std::milli> d2 = t4 - t3;

    std::cout << "map arama:           " << d1.count() << " ms\n";
    std::cout << "unordered_map arama: " << d2.count() << " ms\n";
    std::cout << "Hiz farki: " << d1.count() / d2.count()
              << "x\n";

    return 0;
}
```

Görev 4.1 – Performans Analizi (5 dk)

Programı çalıştırın ve sonuçları kaydedin.
Sorular:

a) `unordered_map` kaç kat daha hızlı?

b) `N` değerini 1000'e düşürün – fark hâlâ belirgin mi? Neden?

Görev 4.2 – stack ile Geri Alma (10 dk)

OBYS prototipine geri alma (undo) mekanizması ekleyin:

a) `std::stack<std::string> islem_gecmisi` tanımlayın

b) Her öğrenci ekleme/silme işlemini stack'e kaydedin
Örnek: `islem_gecmisi.push("Eklendi: 2001 Ali")`

c) `geriAl()` fonksiyonu yazın: son işlemi gösterip stack'ten çıkarsın

d) 3 işlem yapıp 2 kez geri alın, stack'in durumunu yazdırın

Dikkat: `pop()` Değer Döndürmez!

`stack::pop()` sadece elemanı çıkarır, değerini döndürmez. Önce `top()` ile değeri okuyun, sonra `pop()` ile çıkarın.

Görev 4.3 – Bonus: priority_queue ile Acil İşlem Sırası (10 dk)

`std::priority_queue` kullanarak öğrenci taleplerini öncelik sırasına göre işleyen bir sistem yazın:

```
struct Talep {
    int oncelik;  // yuksek = daha acil
    std::string aciklama;
    bool operator<(const Talep& t) const {
        return oncelik < t.oncelik;
    }
};
```

a) 5 talep ekleyin (farklı önceliklerle)

b) Tüm talepleri öncelik sırasında işleyin (çıkarın ve yazdırın)

Teslim Bilgileri

Teslim edilecekler:

1. `calisma1.cpp` – vector görevleri (Görev 1.1–1.3)
2. `calisma2.cpp` – Karşılaştırma tablosu (Görev 2.1–2.2)
3. `calisma3.cpp` – OBYS prototipi (Görev 3.1–3.3)
4. `calisma4.cpp` – unordered_map + stack (Görev 4.1–4.2)
5. *Bonus:* `priority_queue` çözümü (Görev 4.3)

Puanlama: Çalışma 1: 20p Çalışma 2: 20p Çalışma 3: 30p Çalışma 4: 20p Bonus: 10p